

SE Semester Project



Group members:

Ganjina Delawar-046

Muhammad Usama-096

Ammar Javed-021

Abudl Jamil-003

Class: BSCS-4C

Submitted to:

AIMA ZAHOOR

Bahria University Islamabad Campus

Smart inventory System for Small Businesses

0.Existing System Analysis and Problem Identification

0.1 Existing System Overview

Inventory management in small retail shops is commonly handled through manual methods or partially digital solutions. Most businesses rely on notebooks, spreadsheets, or basic Point of Sale (POS) systems that provide limited automation and weak integration between Sales and Inventory operations. As a result, maintaining accurate Product records and stock levels becomes difficult during daily business activities.

This analysis is based on commonly observed practices in small grocery stores and local retail environments. The objective is to examine practical operational issues and identify the limitations that affect Inventory accuracy, Sales tracking, and stock management.

0.2 Operational Environment of Small Retail Shops

Small retail businesses usually operate with limited technical infrastructure and minimal automation. Product handling, stock updates, and Sales recording are often managed manually or through simple software tools that lack centralized control.

Because processes are not standardized, businesses frequently experience inconsistencies in Product naming, Inventory updates, and transaction recording. These operational gaps directly affect stock visibility and decision-making.

0.3 Product Entry Process (Stock Addition)

0.3.1 Process Description

When new stock is received from a Supplier, the shopkeeper or staff records Product information manually or through a digital tool.

In manual environments, Product details are maintained in physical registers or notebooks. Semi-digital environments typically use spreadsheets or basic POS software.

Commonly recorded Product information includes:

- Product name
- Quantity received
- Purchase price
- Supplier details

0.3.2 Observed Issues

Several issues are commonly observed during Product entry:

- Duplicate Product records caused by inconsistent naming

- Missing Product attributes such as purchase price or Supplier information
- Lack of input validation during data entry
- Inconsistent formatting across records

0.3.3 Operational Impact

These problems reduce the accuracy of Product records and complicate Inventory tracking. Over time, inconsistent entries create confusion during stock verification and reporting activities.

0.4 Sales Processing Workflow

0.4.1 Process Description

Sales handling varies depending on the system being used.

In manual systems:

- Only the total bill amount is recorded
- Product-level Sales details are often missing

In POS-based systems:

- Products are scanned or selected digitally
- Sales are recorded per Product
- Inventory is updated automatically after each transaction

0.4.2 Observed Issues

Common operational issues include:

- No real-time Inventory updates in manual systems
- Incorrect Product selection during POS operations
- Wrong quantity input
- Missed or incomplete transaction entries

0.4.3 Operational Impact

These issues create mismatches between recorded and actual Inventory levels, reducing the reliability of stock data and affecting daily operations.

0.5 Inventory Update Practices

0.5.1 Process Description

Inventory updates are generally handled using one of the following approaches:

Manual Approach:

- Physical stock counting at the end of the day or week

Digital Approach:

- Automatic stock updates after Sales transactions

0.5.2 Observed Issues

The following limitations are frequently observed:

- Manual stock counting is time-consuming and prone to human error
- Digital systems depend heavily on accurate transaction input
- Delayed updates reduce Inventory accuracy during busy operations

0.5.3 Operational Impact

Frequent discrepancies occur between actual stock and recorded Inventory. In manual environments, the absence of synchronized Sales and Inventory updates further increases operational inconsistency.

0.6 Low Stock Monitoring Practices

0.6.1 Process Description

Low stock monitoring is usually managed informally by shopkeepers through:

- Memory-based tracking
- Visual inspection of shelves
- Experience-based estimation

0.6.2 Observed Issues

- No automated low stock alert mechanism
- Difficult monitoring for large Product ranges
- Delayed identification of fast-selling Products

0.6.3 Operational Impact

Poor stock monitoring often results in:

- Stockouts and missed Sales opportunities
- Overstocking of low-demand Products
- Increased risk of expiration for perishable items

0.7 Supplier Interaction and Restocking Process

0.7.1 Process Description

Restocking activities are usually performed manually by contacting Suppliers through phone calls or physical visits. Order quantities are generally estimated using observation or prior experience rather than structured analysis.

0.7.2 Observed Issues

- No use of historical Sales data during restocking
- Lack of structured demand planning
- Inconsistent ordering decisions

0.7.3 Operational Impact

These limitations contribute to overstocking or understocking, reducing Inventory efficiency and affecting profitability.

0.8 Audit and Tracking Mechanisms

0.8.1 Process Description

Inventory-related actions such as Product updates, Sales transactions, and stock adjustments are rarely tracked systematically in small retail environments.

0.8.2 Observed Issues

Most systems do not record:

- Who performed the action
- When the action occurred
- Why the modification was made

0.8.3 Operational Impact

The absence of structured tracking reduces accountability and makes it difficult to identify operational errors or unauthorized activities.

0.9 Identified System Limitations

Based on the analysis, the following limitations are identified in existing retail Inventory practices:

- Lack of standardized Product entry procedures
- Inaccurate or delayed Inventory updates
- Limited Product-level Sales tracking
- Absence of automated low stock alerts
- No data-driven restocking mechanism
- Weak audit and activity tracking capabilities

1.0 MILESTONE 1: REQUIREMENTS

1.1 Problem Definition and System Scope

1.1.1 Problem Statement

1.1.1.1 Background Context

Inventory management is a critical operation in small retail businesses, yet many shops still rely on informal and disconnected methods for handling Products, Sales, and stock records. Based on the existing system analysis, most businesses use manual registers, spreadsheets, or basic POS systems with limited integration and control.

Because these approaches lack standardized workflows, maintaining accurate Product information and Inventory records becomes difficult. Errors in Sales recording and delayed stock updates further reduce operational efficiency and create inconsistencies across the system.

1.1.1.2 Stock Mismatch Issue

One of the most common operational problems is stock mismatch, where recorded Inventory does not accurately reflect the physical stock available in the store.

This issue usually occurs due to:

- Unrecorded Sales
- Incorrect quantity entry
- Delayed Inventory updates
- Manual calculation errors

In many manual systems, only the total transaction amount is recorded instead of Product-level Sale information, making accurate stock tracking difficult.

1.1.1.3 Stockout Problem

Stockouts occur when Products become unavailable during customer demand. Since stock monitoring is often based on memory or visual inspection, low-stock Products may remain unnoticed until Inventory is fully depleted.

This affects customer satisfaction and leads to missed Sales opportunities.

1.1.1.4 Overstocking Problem

Overstocking is another major issue caused by weak Inventory monitoring and the absence of structured Sales analysis. Shopkeepers frequently order Products based on estimation instead of actual demand patterns.

As a result:

- Excess capital becomes tied to Inventory

- Storage space is wasted
- Perishable Products face higher risk of expiration or damage

1.1.1.5 Lack of Accountability and Auditability

Current systems provide little or no tracking of Inventory-related actions. Changes to Product records, Inventory quantities, or Sales data are often performed without maintaining proper logs.

Most environments do not record:

- The user performing the action
- The time of modification
- The reason for the change

This makes error identification and operational monitoring difficult.

1.1.2 System Scope Definition

The Smart Inventory System for Small Businesses is designed to address the operational limitations identified in small retail environments. The system focuses on structured Product management, accurate Inventory handling, controlled Sales processing, and activity monitoring while remaining practical and suitable for small business operations.

1.1.3 In-Scope Functional Coverage

The system includes the following core functionalities:

- Product management
- Sale recording
- Real-time Inventory updates
- Low stock monitoring
- Supplier management
- Audit logging
- Inventory reporting
- Role-based User access control

1.1.3.1 Product Management

The system allows authorized users to create and manage Product records, including Product name, category, pricing information, and Supplier association. Structured Product management reduces inconsistent data entry and improves record accuracy.

1.1.3.2 Sales Management

The system records Sales at the Product level, allowing accurate tracking of quantities sold and transaction details for each Sale.

1.1.3.3 Inventory Synchronization

Inventory quantities are updated automatically after successful Sales transactions and stock entry operations, helping maintain accurate stock records during daily operations.

1.1.3.4 Low Stock Monitoring

The system monitors Inventory levels and generates alerts when Product quantities fall below predefined threshold values.

1.1.3.5 Supplier and Restocking Management

The system maintains Supplier information and supports Inventory updates when new stock is received.

1.1.3.6 Audit and Activity Logging

Inventory-related actions such as Product creation, Inventory updates, and Sales transactions are recorded in the AuditLog with timestamps and user identification details.

1.1.3.7 Inventory Visibility and Reporting

Users can view current Inventory levels and access basic Sales and stock reports to support operational monitoring and decision-making.

1.1.3.8 User Access Control

The system applies role-based access control by separating Admin and Staff permissions for different system operations.

1.1.4 Out-of-Scope Features

To maintain feasibility and focus on core Inventory operations, the following features are excluded from the system scope:

- AI-based forecasting and predictive analytics
- Integration with external e-commerce platforms
- Automatic Supplier communication and order placement
- Financial accounting modules (taxation, payroll, profit/loss)
- Multi-branch Inventory synchronization
- Mandatory barcode hardware integration

1.2 Functional Requirements Specifications

1.2.1 Functional Requirements

This section defines the operational capabilities of the Smart Inventory System based on the workflows and issues identified during system analysis. The requirements maintain consistency with the system entities: Product, Inventory, Sale, User, Supplier, and AuditLog.

1.2.2 Product Management Requirements

1.2.2.1 Product Creation

FR-01: Authorized users can add a new Product with details including Product name, category, purchase price, selling price, and Supplier information.

1.2.2.2 Duplicate Product Validation

FR-02: Duplicate Product entries are restricted through validation of Product name and identifier during creation.

1.2.2.3 Product Update

FR-03: Existing Product information, including pricing and Supplier details, can be modified by authorized users.

1.2.3 Inventory Management Requirements

1.2.3.1 Stock Entry and Update

FR-04: Users can record incoming stock and update Inventory quantities for associated Products.

1.2.3.2 Real-Time Inventory Deduction

FR-05: Sold quantities are automatically deducted from Inventory immediately after successful Sale recording.

1.2.3.3 Inventory Availability Validation

FR-06: Sale processing is blocked when the requested quantity exceeds available Inventory.

1.2.3.4 Inventory Monitoring

FR-07: Users can view current Inventory levels for all Products in real time.

1.2.4 Sales Management Requirements

1.2.4.1 Sale Recording

FR-08: Sales transactions are recorded at the Product level with Product ID, quantity sold, selling price, and timestamp details.

1.2.5 Low Stock Monitoring Requirements

1.2.5.1 Low Stock Alert Generation

FR-09: Low stock alerts are generated whenever Inventory quantity falls below the predefined threshold value.

1.2.5.2 Threshold Configuration

FR-10: Users can define and update low stock threshold values for individual Products.

1.2.6 Supplier Management Requirements

1.2.6.1 Supplier Association

FR-11: Supplier information can be recorded and linked with Products for restocking and tracking purposes.

1.2.6.2 Stock Update from Supplier

FR-12: Inventory quantities can be updated when new stock is received from a Supplier.

1.2.7 Audit and Logging Requirements

1.2.7.1 Audit Log Maintenance

FR-13: Inventory-related activities, including Product updates, Inventory changes, and Sales transactions, are recorded in the AuditLog.

1.2.7.2 Audit Log Details

FR-14: Each AuditLog entry stores user identification, action type, and timestamp information.

1.2.8 User Management and Access Control Requirements

1.2.8.1 User Account Management

FR-15: Admin users can create and manage User accounts with assigned system roles.

1.2.8.2 Role-Based Access Restriction

FR-16: System functionalities and permissions are controlled according to assigned User roles (Admin and Staff).

1.2.9 Reporting and Retrieval Requirements

1.2.9.1 Reporting

FR-17: Users can generate and view basic Sales and Inventory reports for operational monitoring.

1.2.9.2 Product Search and Retrieval

FR-18: Product information can be searched and retrieved using Product name or identifier.

1.3 Non-Functional Requirements Specification

1.3.1 Non-Functional Requirements

This section defines the quality attributes and operational constraints of the Smart Inventory System. These requirements focus on performance, security, usability, reliability, and operational stability.

1.3.2 Performance Requirements

1.3.2.1 Response Time Constraint

NFR-01: User operations such as Product entry, Sale recording, and Inventory updates should complete within 2 seconds under normal operating conditions.

1.3.2.2 Concurrent User Handling

NFR-02: The system should support at least 5 concurrent users without increasing response time beyond 3 seconds.

1.3.3 Security Requirements

1.3.3.1 User Authentication

NFR-03: Access to the system requires authentication using a valid username and password.

1.3.3.2 Role-Based Access Control

NFR-04: User permissions and accessible operations are restricted according to assigned roles.

1.3.3.3 Data Encryption

NFR-05: Sensitive data, particularly user credentials, must remain stored in encrypted form.

1.3.3.4 Audit Logging for Security

NFR-06: Critical activities such as Product updates, Inventory changes, and Sales transactions must be recorded with user identification and timestamps.

1.3.4 Usability Requirements

1.3.4.1 Ease of Learning

NFR-07: New users should be able to perform basic operations such as Product entry and Sale recording with minimal training.

1.3.4.2 Transaction Efficiency

NFR-08: Completing a Sale transaction should require no more than five operational steps.

1.3.4.3 Error Handling and Feedback

NFR-09: Clear and meaningful error messages must be displayed for invalid inputs and failed operations.

1.3.5 Reliability Requirements

1.3.5.1 Data Consistency

NFR-10: Inventory and Sales records must remain synchronized to prevent incorrect stock values.

1.3.5.2 System Availability

NFR-11: Operational availability should remain at or above 99% during business hours.

1.3.5.3 Data Persistence and Recovery

NFR-12: Transactions and operational data should be saved automatically to reduce data loss during unexpected failures.

1.4 Use Case Modeling

1.4.1 Use Case Modeling Overview

This section defines how external actors interact with the Smart Inventory System through structured use cases. The model represents operational workflows related to Product management, Inventory handling, Sales processing, Supplier management, and User control.

All use cases remain directly connected to the Functional Requirements to maintain consistency across system modeling, design, and testing.

1.4.2 Actor Identification

Actors represent external entities that interact with the system. They are categorized according to their operational responsibilities and level of interaction.

1.4.2.1 Primary Actors

❖ Admin

Responsible for managing Products, Users, Suppliers, system configurations, reports, and monitoring activities recorded in the AuditLog.

❖ Staff

Handles daily operational tasks such as recording Sales, updating Inventory, checking stock levels, and viewing reports.

❖ Supplier

Provides Products during restocking operations. Supplier interaction remains indirect through Inventory and Product updates performed by system users.

1.4.2.2 Supporting Actors

❖ Notification Service

Processes and delivers low stock alerts generated by the system.

❖ Authentication Module

controls access permissions according to assigned roles.

1.4.3 Use Case Model Overview

The Use Case Model represents the major functionalities of the Smart Inventory System and the interactions between actors and system processes. Each use case corresponds to one or more Functional Requirements.

1.4.4 Use Case Identification and Traceability

1.4.4.1 Core Operational Use Cases

- Manage Product (FR-01, FR-02, FR-03)
- Record Sale (FR-05, FR-06, FR-08)
- Update Inventory (Stock Entry) (FR-04, FR-12)
- View Inventory (FR-07)

1.4.4.2 Monitoring and Control Use Cases

- Manage Low Stock Alert (FR-09, FR-10)
- Maintain Audit Log (FR-13, FR-14)

1.4.4.3 Administrative Use Cases

- Manage Supplier (FR-11)
- Manage Users (FR-15, FR-16)

1.4.4.4 Reporting and Retrieval Use Cases

- View Reports (FR-17)
- Search Product (FR-18)

1.4.5 Actor–Use Case Interaction Description

1.4.5.1 Admin Interaction

The Admin performs supervisory and management-level operations within the system. Responsibilities include Product management, Supplier maintenance, User control, threshold configuration, and monitoring reports and audit activities.

1.4.5.2 Staff Interaction

The Staff actor handles routine operational activities such as Sales recording, Inventory updates, and stock monitoring. Access to features is controlled according to assigned permissions.

1.4.5.3 Supplier Interaction

Suppliers participate indirectly in the restocking workflow. Supplier-related information is associated with Products and Inventory updates during stock entry operations.

1.4.5.4 Supporting System Interaction

The Notification Service delivers low stock alerts when Inventory quantities fall below threshold levels. The Authentication Module verifies user credentials and enforces role-based access restrictions. The System Database manages storage and retrieval of operational data.

1.4.6 Use Case Relationships

Use case relationships define dependencies and conditional interactions between different system functions.

1.4.6.1 Include Relationships

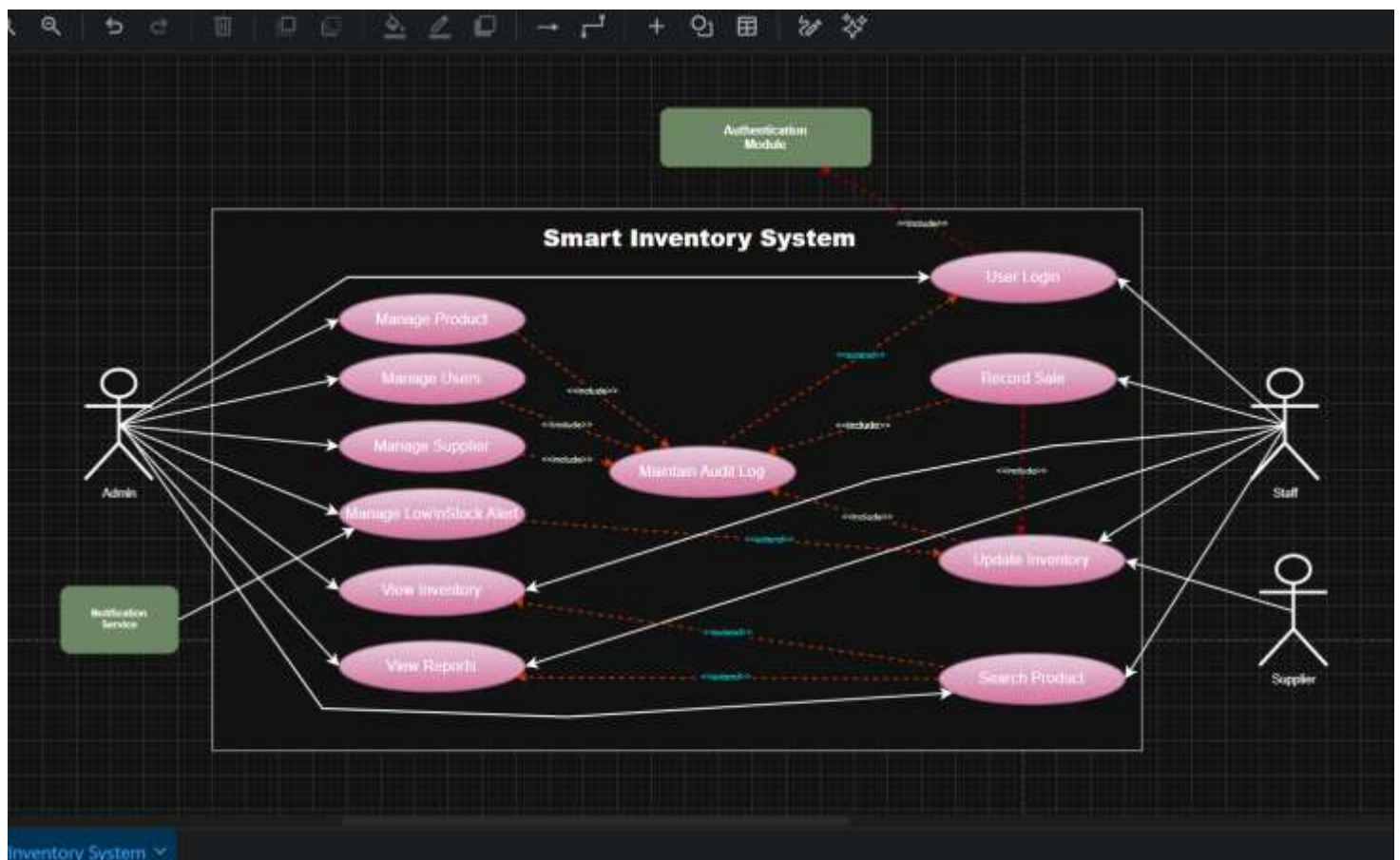
- ❖ <<include>> relationships are for dependent processes
 - User Login <<include>> Authentication Module
(Authentication is mandatory before accessing the system)
 - Record Sale <<include>> Update Inventory
(Inventory must update after every sale)
 - Record Sale <<include>> Maintain Audit Log
(System must record completed sale transactions)
 - Manage Product <<include>> Maintain Audit Log
(System must record product-related changes)
 - Manage Users <<include>> Maintain Audit Log
(System must record user account modifications)
 - Manage Supplier <<include>> Maintain Audit Log
(System must record supplier information changes)
 - Update Inventory <<include>> Maintain Audit Log
(System must record inventory stock changes)

1.4.6.2 Extend Relationships

- ❖ <<extend>> relationships for conditional behavior
 - Manage Low Stock Alert <<extend>> Update Inventory
(Triggered when Inventory quantity falls below threshold)
 - Search Product <<extend>> View Inventory
(Triggered when user searches for a specific product while viewing inventory)
 - Search Product <<extend>> View Reports
(Triggered when reports require product-specific filtering)
 - sMaintain Audit Log <<extend>> User Login
(Triggered when failed or suspicious login attempts occur)

1.4.7 Use Case Model Representation:

Diagram



1.4.8 Detailed Use Case Descriptions

Detailed use case specifications define the execution flow, conditions, and outcomes of each major system operation.

1.4.8.1 Use Case 1: Add Product

Field	Description
Name	Add Product
Actor	Admin
Preconditions	Admin is authenticated
Main Flow	1. Admin selects “Add Product”. 2. Product details are entered. 3. Input validation is performed. 4. Duplicate Product validation is checked. 5. Product data is stored. 6. Action is recorded in AuditLog.
Alternate Flow	If duplicate Product data exists, an error message is displayed and saving is cancelled.
Postconditions	Product becomes available in the Inventory system.
Mapped Requirements	FR-01, FR-02, FR-13, FR-14

1.4.8.2 Use Case 2: Update Product

Field	Description
Name	Update Product
Actor	Admin
Preconditions	Product exists and Admin is authenticated
Main Flow	1. Admin searches for a Product. 2. Product details are modified. 3. Validation is performed. 4. Updated information is saved. 5. Action is recorded in AuditLog.
Alternate Flow	Invalid input results in an error message and update rejection.

Postconditions	Product information is updated successfully.
Mapped Requirements	FR-03, FR-13, FR-14

1.4.8.3 Use Case 3: Record Sale

Field	Description
Name	Record Sale
Actor	Staff
Preconditions	Staff is authenticated, Product exists, and Inventory is sufficient
Main Flow	1. Staff selects “Record Sale”. 2. Product and quantity are entered. 3. Inventory availability is validated. 4. Sale record is created. 5. Inventory quantity is updated. 6. Action is logged in AuditLog.
Alternate Flow	If stock is insufficient, the transaction is cancelled and an error message is displayed.
Postconditions	Sale and Inventory records are updated successfully.
Mapped Requirements	FR-05, FR-06, FR-08, FR-13, FR-14

1.4.8.4 Use Case 4: Update Inventory (Stock Entry)

Field	Description
Name	Update Inventory (Stock Entry)
Actor	Admin / Staff
Preconditions	Product exists and User is authenticated
Main Flow	1. User selects Product. 2. Quantity received is entered. 3. Inventory quantity is updated. 4. Action is recorded in AuditLog.
Alternate Flow	Invalid quantities are rejected with an error message.

Postconditions	Inventory quantity is increased successfully.
Mapped Requirements	FR-04, FR-12, FR-13, FR-14

1.4.8.5 Use Case 5: View Inventory

Field	Description
Name	View Inventory
Actor	Admin / Staff
Preconditions	User is authenticated
Main Flow	1. User selects “View Inventory”. 2. Inventory data is retrieved. 3. Current stock levels are displayed.
Alternate Flow	If no Products exist, an empty Inventory message is shown.
Postconditions	Current Inventory status becomes visible to the user.
Mapped Requirements	FR-07

1.4.8.6 Use Case 6: Manage Low Stock Alert

Field	Description
Name	Manage Low Stock Alert
Actor	Admin
Preconditions	Product exists and Admin is authenticated
Main Flow	1. Admin selects a Product. 2. Low stock threshold value is entered or updated. 3. Threshold information is saved. 4. Inventory levels are monitored against the defined threshold. 5. Alert is triggered when stock falls below the limit.
Alternate Flow	Invalid threshold values generate an error message.
Postconditions	Low stock monitoring becomes active for the selected Product.
Mapped Requirements	FR-09, FR-10

1.4.8.7 Use Case 7: Manage Supplier

Field	Description
Name	Manage Supplier
Actor	Admin
Preconditions	Admin is authenticated
Main Flow	1. Admin selects “Add/Update Supplier”. 2. Supplier details are entered or modified. 3. Input validation is performed. 4. Supplier information is stored successfully.
Alternate Flow	Missing or invalid fields result in validation errors.
Postconditions	Supplier information becomes available for Product association and restocking operations.
Mapped Requirements	FR-11

1.4.8.8 Use Case 8: Manage Users

Field	Description
Name	Manage Users
Actor	Admin
Preconditions	Admin is authenticated
Main Flow	1. Admin selects “Manage Users”. 2. User account information is entered or updated. 3. System role is assigned. 4. User data is saved successfully.
Alternate Flow	Duplicate usernames generate an error message and prevent account creation.
Postconditions	User accounts are created or updated with assigned permissions.
Mapped Requirements	FR-15, FR-16

1.4.8.9 Use Case 9: View Reports

Field	Description
Name	View Reports
Actor	Admin / Staff
Preconditions	User is authenticated
Main Flow	1. User selects “View Reports”. 2. Sales and Inventory records are retrieved. 3. Requested report is generated and displayed.
Alternate Flow	If no records are available, the system displays “No records found”.
Postconditions	Users can review Sales and Inventory information for monitoring purposes.
Mapped Requirements	FR-17

1.4.8.10 Use Case 10: Search Product

Field	Description
Name	Search Product
Actor	Admin / Staff
Preconditions	User is authenticated
Main Flow	1. User enters Product name or identifier. 2. Search request is processed. 3. Matching Product records are retrieved. 4. Results are displayed to the user.
Alternate Flow	If no matching Product exists, the message “Product not found” is displayed.
Postconditions	Requested Product information becomes accessible to the user.
Mapped Requirements	FR-18

1.5 Requirements Traceability Matrix

1.5.1 Requirements Traceability Overview

The Requirements Traceability Matrix (RTM) establishes relationships between Functional Requirements, Use Cases, system models, and Test Cases. This mapping helps verify that every requirement is represented in system behavior, design models, and validation activities.

1.5.2 Traceability Table

Requirement ID	Use Case	Diagram	Test Case
FR-01	Add Product	Use Case Model, Class Structure Model	TC-01
FR-02	Add Product	Use Case Model, Class Structure Model	TC-02
FR-03	Update Product	Use Case Model, Class Structure Model	TC-03
FR-04	Update Inventory (Stock Entry)	DFD Level 1, Activity Model	TC-04
FR-05	Record Sale	Sequence Model, DFD Level 1	TC-05
FR-06	Record Sale	Sequence Model, Activity Model	TC-06
FR-07	View Inventory	DFD Level 0	TC-07
FR-08	Record Sale	Sequence Model	TC-08
FR-09	Manage Low Stock Alert	Process Workflow Model, Data Flow Model Level 0	TC-09
FR-10	Manage Low Stock Alert	Process Workflow Model	TC-10
FR-11	Manage Supplier	Class Structure Model, Use Case Model	TC-11
FR-12	Update Inventory (Stock Entry)	DFD Level 1, Activity Model	TC-12
FR-13	Maintain Audit Log	Class Structure Model, Sequence Model	TC-13
FR-14	Maintain Audit Log	Sequence Model	TC-14
FR-15	Manage Users	Use Case Model, Class Structure Model	TC-15
FR-16	Manage Users	Use Case Model	TC-16
FR-17	View Reports	DFD Level 0	TC-17
FR-18	Search Product	Use Case Model, Sequence Model	TC-18

1.5.3 Traceability Coverage

The traceability structure provides complete linkage between requirements, system models, and validation activities.

Coverage includes:

- Mapping of every Functional Requirement to at least one Use Case
- Representation of each requirement in system models
- Corresponding Test Cases for validation

This structure reduces the possibility of missing functionality during implementation and testing.

1.5.4 Cross-Section Consistency

The traceability defined in the RTM remains consistent across all major project phases.

❖ Modeling Phase

- Data Flow Diagrams (DFD)
- Use Case Model
- Sequence Model
- Activity Model

❖ Design Phase

- Class Structure Model
- Database Design

❖ Testing Phase

- Test Case Specification
- Validation Analysis

2.0 MILESTONE 2: SYSTEM MODELING

2.1 Context-Level Data Flow Model (DFD Level 0)

2.1.1 Context-Level Overview

The Context-Level Data Flow Model represents the Smart Inventory System as a single high-level process and shows how external entities interact with the system through data exchange.

At this stage, internal processing details are not shown. The model focuses only on system boundaries, external actors, and major input/output flows.

2.1.2 System Boundary Description

The Smart Inventory System operates as a centralized process responsible for handling Product management, Inventory operations, Sales processing, User management, reporting, and activity logging.

Internal validation logic, database operations, and business rules remain within the system boundary and are not represented in this model.

2.1.3 External Entities

❖ Admin

Provides Product information, User data, Supplier details, and configuration inputs. Receives reports, confirmations, and system responses.

❖ Staff

Records Sales transactions, performs Inventory updates, and accesses stock information and reports.

❖ Supplier

Acts as the external source of Products during restocking activities. Supplier-related information is recorded through system operations.

❖ Notification Service

Receives alert triggers from the system and delivers notifications such as low stock alerts.

2.1.4 Data Flow Description

❖ Input Data

- Product Data (from Admin)
- User Data (from Admin)
- Sales Data (from Staff)

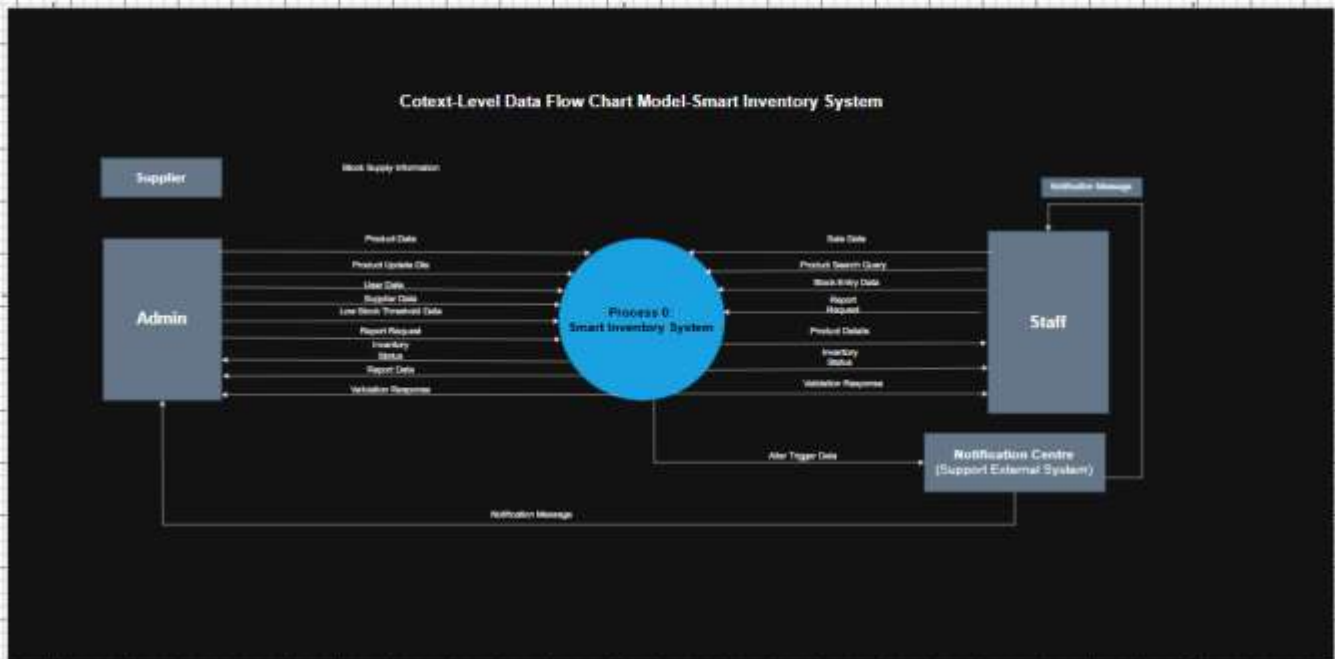
- Stock Entry Data (from Admin/Staff)
- Supplier Information

❖ Output Data

- Inventory Status
- Reports
- Validation Responses
- Alert Notifications

2.1.5 Context-Level DFD Representation:

Diagram



2.2 Data Flow Modeling

2.2.1 Data Flow Modeling Overview

Data Flow Modeling represents how information moves through the Smart Inventory System by identifying major processes, data stores, and interactions between system components and external entities. The model provides a structured representation of operational workflows and supports accurate system design.

2.2.2 DFD Level 0: System Decomposition Model

2.2.2.1 Overview

The Level 0 DFD expands the Context-Level Model by decomposing the Smart Inventory System into major internal processes while preserving interactions with external entities and data stores.

2.2.2.2 Major Processes

The system is divided into the following core processes:

- Manage Product
- Process Sale
- Manage Inventory
- Manage Users
- Generate Reports & Alerts

2.2.2.3 Data Stores

The following internal data stores are included:

- Product Data Store
- Inventory Data Store
- Sales Data Store
- User Data Store
- Supplier Data Store
- AuditLog Data Store

2.2.2.4 External Entities

- Admin
- Staff
- Supplier

- Notification Service

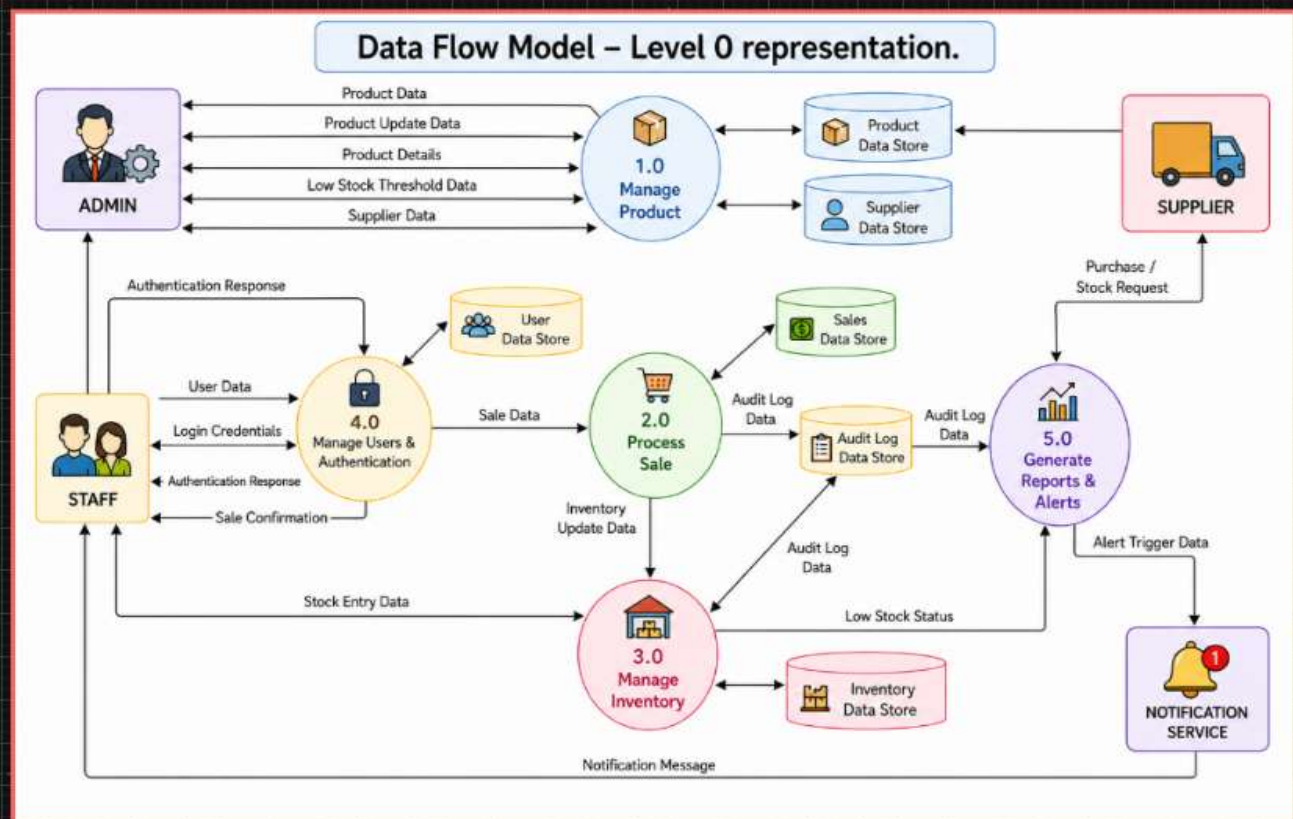
2.2.2.5 Data Flow Description

Major system data flows include:

- Product, User, and Supplier data provided by Admin
- Sales and Inventory updates submitted by Staff
- Restocking information associated with Suppliers
- Process interactions with internal data stores
- Report and Inventory information returned to users
- Alert notifications sent to the Notification Service

2.2.3 DFD Level 0 Representation

Diagram



2.2.4 DFD Level 1: Sale Processing Model

2.2.4.1 Overview

The Level 1 DFD provides a detailed decomposition of the Process Sale module. Since Sales transactions directly affect Inventory records, this process requires multiple validation and update operations.

2.2.4.2 Sub-Processes

The Sale process is divided into the following sub-processes:

- Receive Sale Input
- Validate Product & Stock Availability
- Record Sale Transaction
- Update Inventory
- Log Transaction in AuditLog

2.2.4.3 Data Stores Used

- Product Data Store
- Inventory Data Store
- Sales Data Store
- AuditLog Data Store

2.2.4.4 External Entity

- Staff

2.2.4.5 Data Flow Description

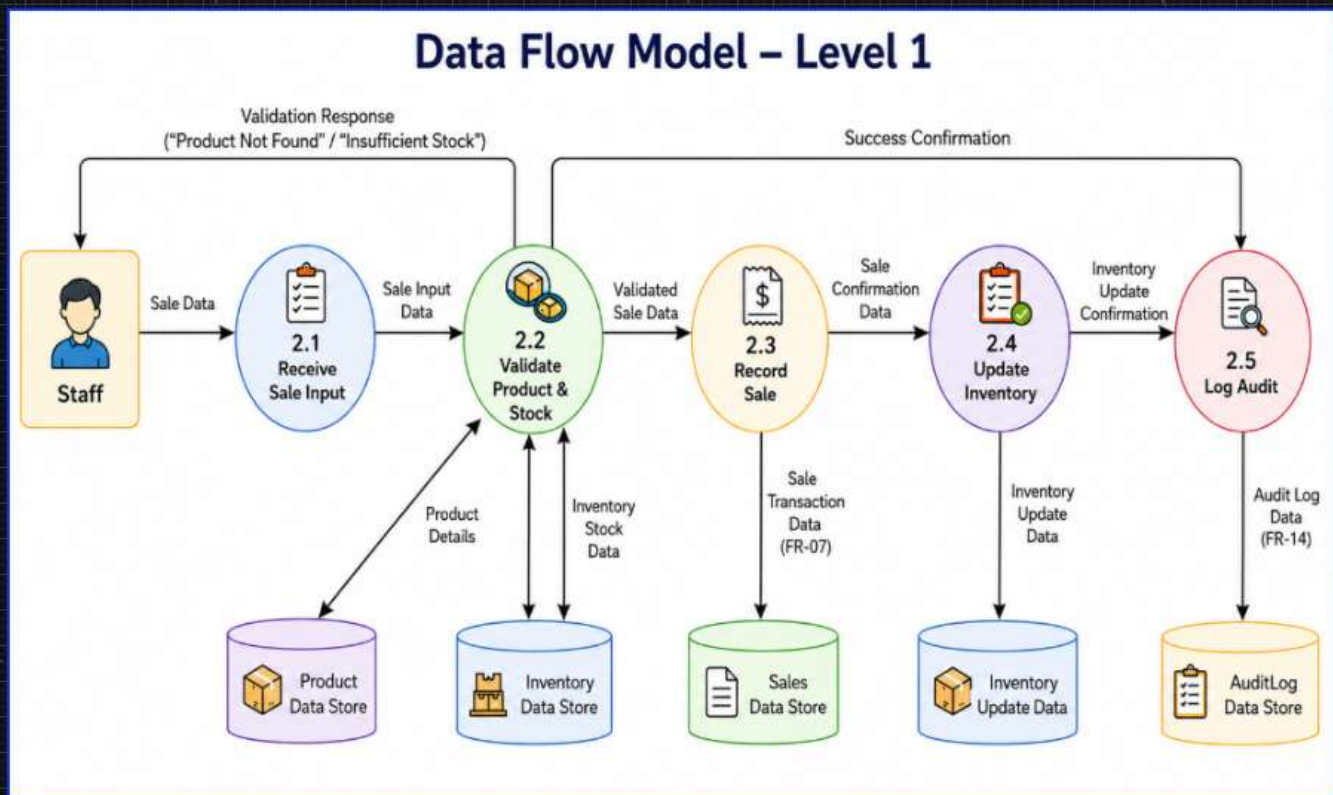
The Sale processing workflow follows this sequence:

- Staff submits Sale information
- Product and Inventory data are retrieved for validation
- If validation succeeds:
 - Sale transaction is recorded
 - Inventory quantity is updated
 - AuditLog entry is created
 - Confirmation message is returned

- If validation fails:
 - Error message is displayed
 - Transaction processing stops

2.2.5 DFD Level 1 Representation:

Diagram



2.3 Process Flow Model: Sale Transaction Workflow

2.3.1 Workflow Overview

The Process Flow Model represents the execution logic of a Sale transaction within the Smart Inventory System. Unlike DFDs, which focus on data movement, this model emphasizes control flow, validation steps, and operational decisions during Sale processing.

The workflow is derived directly from the Sale Processing module and reflects the actual sequence of activities performed during a transaction.

2.3.2 Sale Transaction Workflow

The process begins when a Staff member initiates a Sale by entering Product and quantity details.

The system first validates whether the Product exists in the database. If the Product is invalid or unavailable, the transaction stops and an error message is displayed.

If Product validation succeeds, Inventory availability is checked. Transactions requesting quantities greater than available stock are rejected to prevent incorrect Inventory values.

After successful validation:

- The Sale transaction is recorded
- Inventory quantity is updated
- AuditLog entry is created
- Confirmation message is displayed

2.3.3 Step-by-Step Workflow

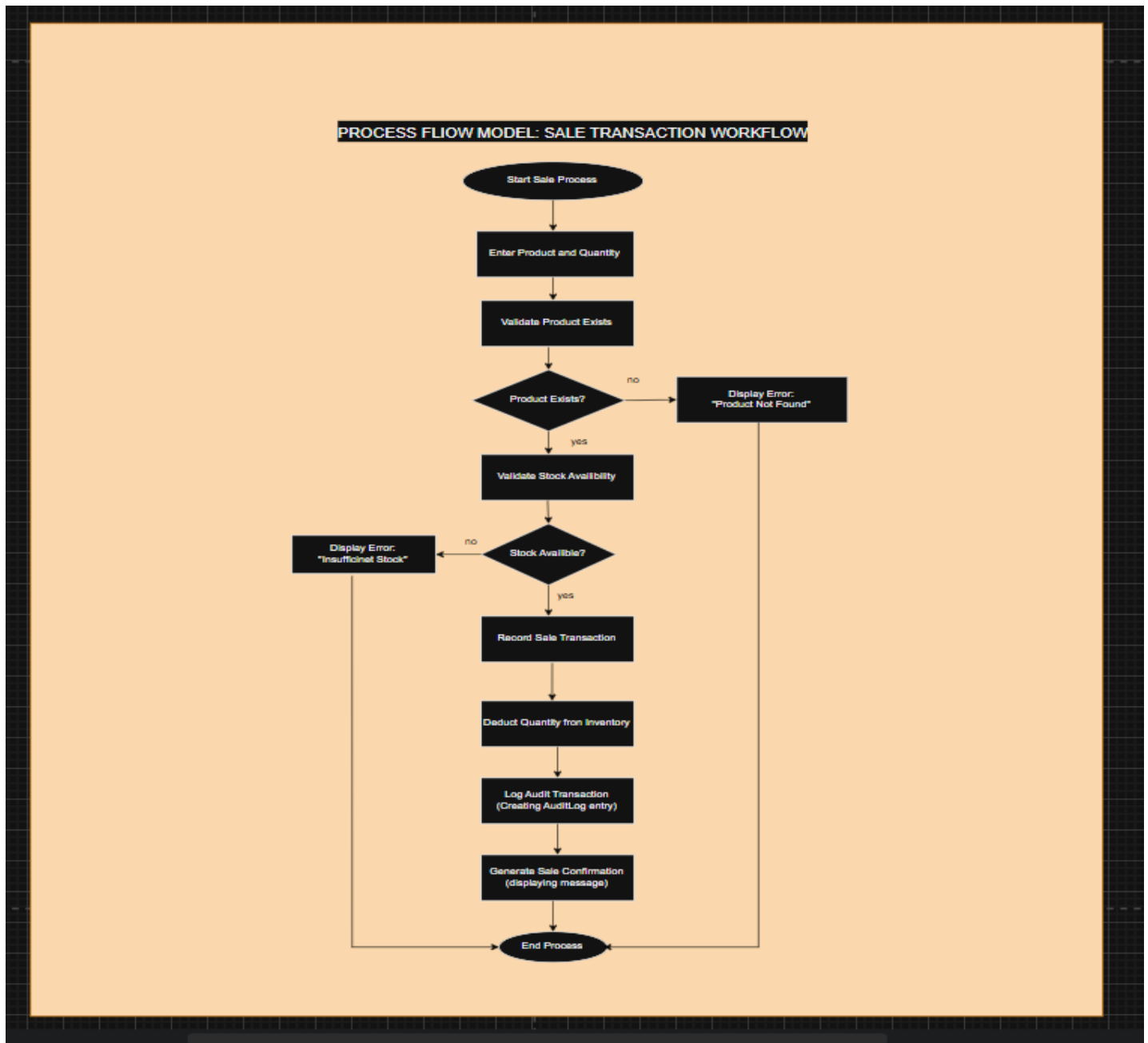
- 1) Start
- 2) Staff enters Product and quantity
- 3) Validate Product existence
- 4) If Product does not exist → Display error → End
- 5) Validate Inventory availability
- 6) If stock is insufficient → Display “Insufficient Stock” → End
- 7) Record Sale transaction
- 8) Update Inventory quantity
- 9) Record action in AuditLog
- 10) Display confirmation message
- 11) End

2.3.4 Requirement Mapping

The workflow directly corresponds to the following Functional Requirements:

- FR-05: Real-Time Inventory Deduction
- FR-06: Inventory Availability Validation
- FR-08: Sale Recording

2.3.5 Process Flow Model Representation: Diagram



2.3.6 Workflow Significance

The Sale workflow contains multiple validation and update operations that directly influence Inventory accuracy and transaction reliability.

Visual representation of this process improves implementation clarity, reduces ambiguity in execution flow, and simplifies validation during development and testing.

2.4 Data Dictionary Specification

2.4.1 Data Dictionary Overview

The Data Dictionary defines the major data elements used throughout the Smart Inventory System. It standardizes data representation across processes, models, and database structures to maintain consistency during design and implementation.

2.4.2 Data Elements Table

Data Name	Description	Source	Destination
Product Data	Product details including name, category, prices, and Supplier ID	Admin	Product Data Store
Product Update Data	Modified Product information	Admin	Product Data Store
Product Search Query	Product name or identifier used for retrieval	Admin / Staff	System
Product Details	Retrieved Product information	System	Admin / Staff
Inventory Data	Current stock quantity for Products	System	Admin / Staff
Stock Entry Data	Quantity received during restocking	Admin / Staff	Inventory Data Store
Sale Data	Product ID, quantity sold, price, and timestamp	Staff	Sales Data Store
Inventory Update Data	Updated stock quantity after Sales or restocking	System	Inventory Data Store
User Data	Username, password, and role information	Admin	User Data Store
Authentication Data	Login credentials entered during authentication	Admin / Staff	Authentication Module
Supplier Data	Supplier information and contact details	Admin	Supplier Data Store
Report Request	Request for Sales or Inventory reports	Admin / Staff	System
Report Data	Generated report information	System	Admin / Staff

Low Stock Threshold Data	Minimum stock limit for Products	Admin	System
Alert Trigger Data	Trigger generated when stock falls below threshold	System	Notification Service
Notification Message	Low stock alert notification	Notification Service	Admin / Staff
AuditLog Data	User actions with timestamps and action details	System	AuditLog Data Store
Validation Response	Success or error response after processing	System	Admin / Staff

2.4.3 Consistency and Alignment

The data elements defined in the dictionary remain consistent with:

- Functional Requirements
- Data Flow Models
- Use Cases
- Class Structure Model
- Database Design

Each data element corresponds directly to system processes and operational workflows.

2.4.4 Process Alignment

The following data elements are associated with major system operations:

❖ Sale Processing

- Sale Data
- Inventory Update Data

❖ Inventory Management

- Stock Entry Data

- Inventory Data

❖ **Product Management**

- Product Data
- Product Update Data

❖ **Security and Monitoring**

- Authentication Data
- AuditLog Data
- Validation Response

2.5 Structural Design Model: Class Architecture

2.5.1 Class Architecture Overview

The Class Architecture represents the structural design of the Smart Inventory System using object-oriented concepts. It identifies the core classes, their attributes, methods, and relationships based on system requirements and operational workflows.

The design remains consistent with the Data Dictionary, Functional Requirements, Use Cases, and database structure.

2.5.2 Core Classes

The system is centered around the following primary classes:

- Product
- Inventory
- User
- Supplier
- Sale
- AuditLog

2.5.3 Class Definitions

2.5.3.1 Class: Product

<i>Attribute</i>	<i>Data Type</i>
------------------	------------------

<i>productID</i>	int
<i>name</i>	string
<i>category</i>	string
<i>purchasePrice</i>	float
<i>sellingPrice</i>	float
<i>supplierID</i>	int

❖ Methods

- addProduct()
- updateProduct()
- getProductDetails()

The Product class manages Product-related information and operations including creation, updates, and retrieval.

2.5.3.2 Class: Inventory

<i>Attribute</i>	<i>Data Type</i>
<i>inventoryID</i>	int
<i>productID</i>	int
<i>quantity</i>	int
<i>lowStockThreshold</i>	Int

❖ Methods

- updateStock(quantity)
- deductStock(quantity)
- checkLowStock()

The Inventory class maintains stock quantities and handles Inventory-related operations.

2.5.3.3 Class: User

<i>Attribute</i>	<i>Data Type</i>
------------------	------------------

<i>userID</i>	int
<i>username</i>	string
<i>password</i>	string
<i>role</i>	string

❖ **Methods**

- login()
- createUser()
- updateUser()

The User class controls authentication and role-based access operations.

2.5.3.4 Class: Supplier

<i>Attribute</i>	<i>Data Type</i>
<i>supplierID</i>	int
<i>name</i>	string
<i>contactInfo</i>	string

❖ **Methods**

- addSupplier()
- updateSupplier()

The Supplier class stores Supplier-related information associated with Product sourcing and restocking.

2.5.3.5 Class: Sale

<i>Attribute</i>	<i>Data Type</i>
<i>saleID</i>	int
<i>productID</i>	int
<i>quantity</i>	int
<i>totalPrice</i>	float
<i>timestamp</i>	datetime

❖ **Methods**

- recordSale()

- calculateTotal()

The Sale class manages transaction records and Sale processing operations.

2.5.3.6 Class: AuditLog

<i>Attribute</i>	<i>Data Type</i>
<i>logID</i>	int
<i>userID</i>	int
<i>actionType</i>	string
<i>timestamp</i>	datetime

❖ Methods

- logAction()
- viewLogs()

The AuditLog class records system activities and operational events.

2.5.4 Class Relationships

2.5.4.1 Association Relationships

❖ Product ↔ Inventory (1:1)

Each Product is associated with one Inventory record responsible for maintaining stock information.

❖ Product ↔ Supplier (1:M)

A Supplier can provide multiple Products, while each Product is linked to one Supplier.

❖ User ↔ AuditLog

User activities are recorded within the AuditLog for operational monitoring.

❖ Sale ↔ Product

Each Sale transaction references a specific Product.

2.5.4.2 Dependency Relationships

❖ Sale → Inventory

Sale operations depend on Inventory updates after successful transaction processing.

❖ Sale → AuditLog

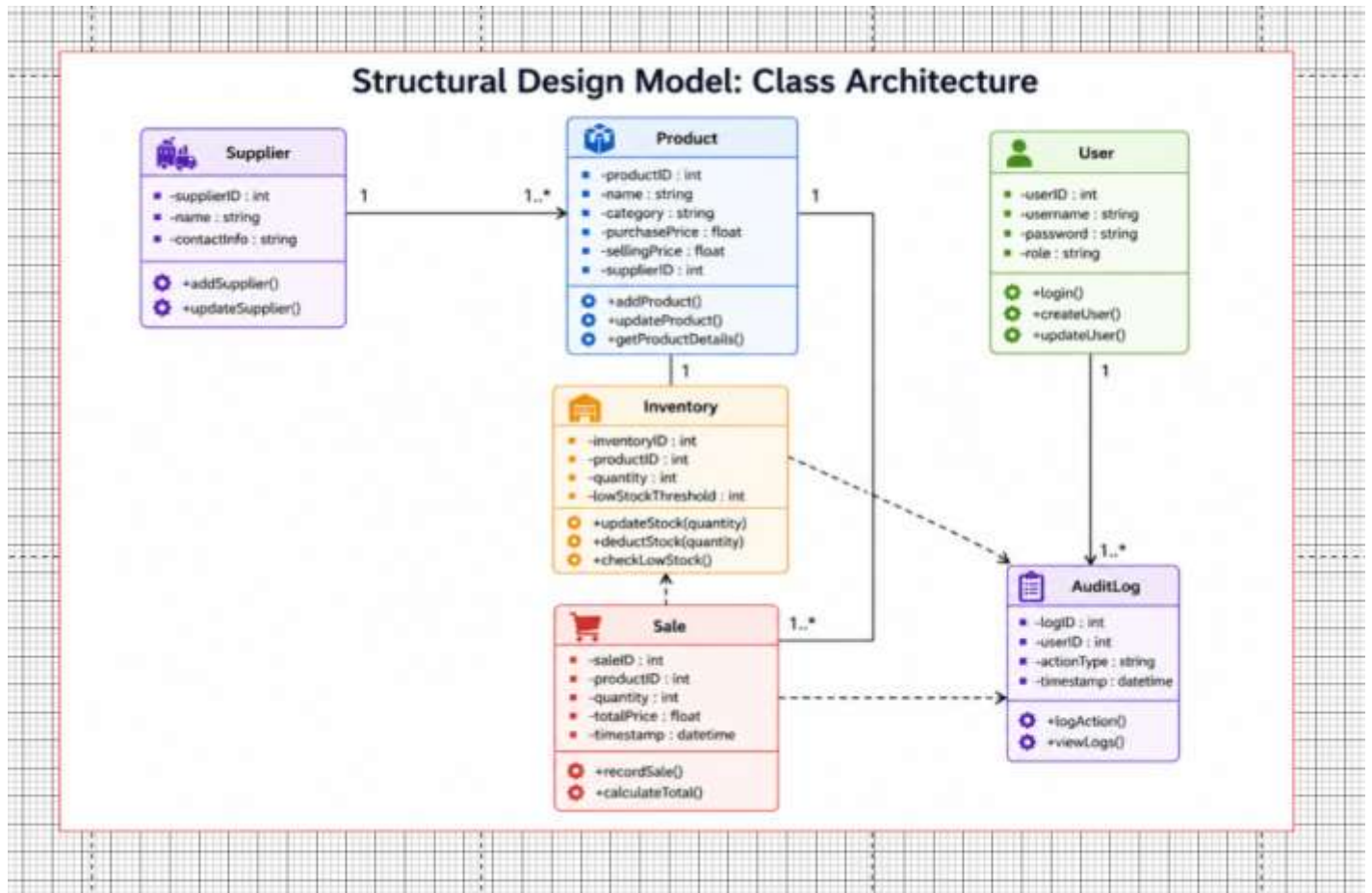
Each completed Sale operation generates an AuditLog entry.

❖ Inventory → AuditLog

Inventory modifications are recorded within the AuditLog.

2.5.5 Class Architecture Representation

Diagram



2.6 Behavioral and Interaction Modeling

2.6.1 Behavioral Modeling Overview

Behavioral and Interaction Modeling represents how system components interact during execution and how operational workflows are processed internally.

The models in this section focus primarily on Sale processing because it directly affects Inventory accuracy, transaction validation, and operational reliability.

2.6.2 Interaction Model: Sequence Flow for Sale Processing

2.6.2.1 Interaction Overview

The Sequence Model illustrates the order of interactions between system components during a Sale transaction. It shows how requests move through the UI, control layer, entities, and database operations.

2.6.2.2 Requirement Mapping

The interaction flow corresponds to:

- FR-05: Real-Time Inventory Deduction
- FR-06: Inventory Availability Validation
- FR-08: Sale Recording
- FR-13: Audit Log Maintenance
- FR-14: Audit Log Details

2.6.2.3 Interaction Description

The process starts when a Staff member initiates a Sale through the user interface.

The request is forwarded to the Controller, which coordinates the operational flow. Product validation is performed first, followed by Inventory availability checking.

If validation succeeds:

- Sale record is created
- Inventory quantity is updated
- AuditLog entry is generated
- Database changes are saved
- Confirmation message is returned

If validation fails, processing stops and an error message is displayed.

2.6.2.4 Interaction Flow

1. Staff → UI (Initiate Sale)

2. UI → Controller (Forward Request)
3. Controller → Product (Validate Product)
4. Controller → Inventory (Check Stock Availability)
5. Decision: Stock Available?

If No:

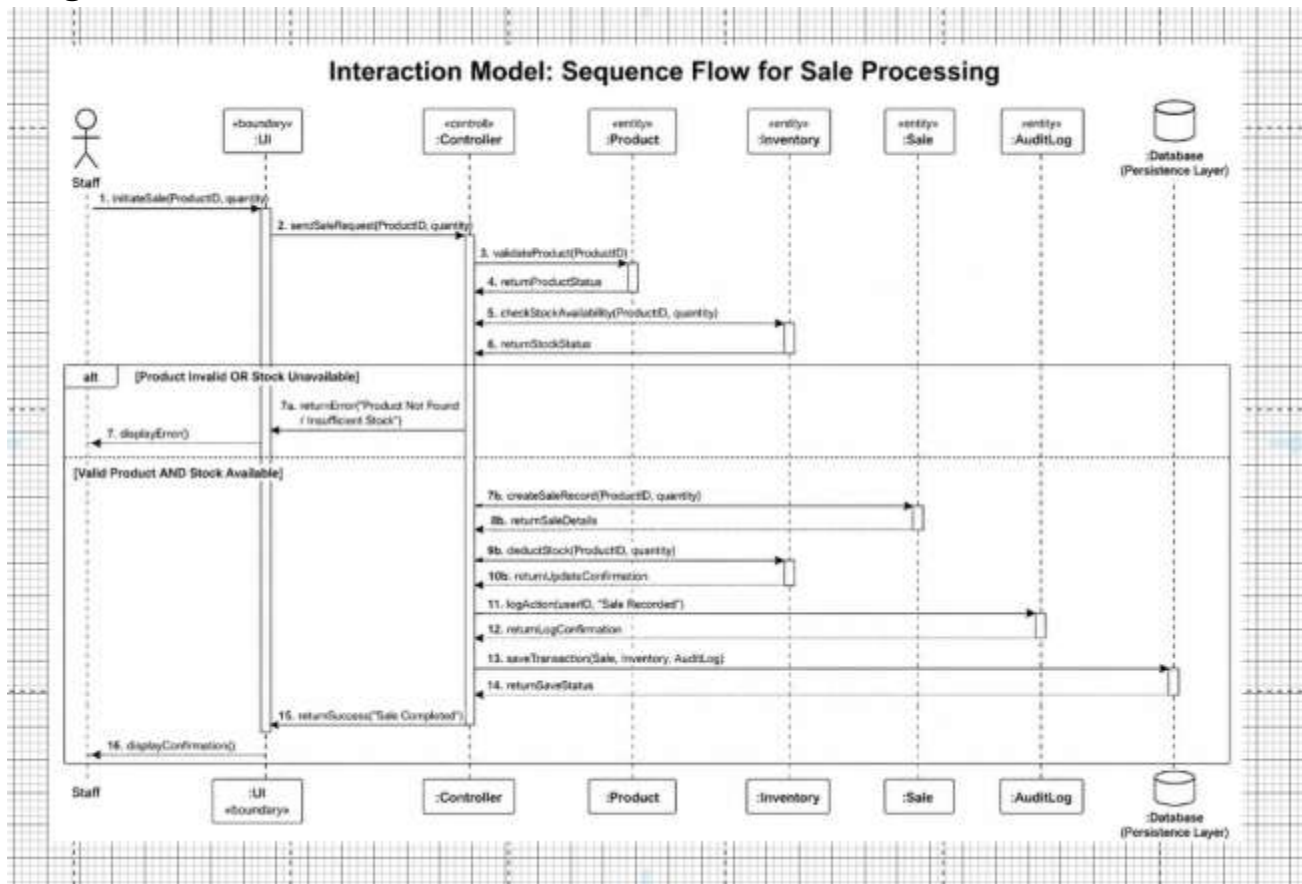
Controller → UI (Display Error Message)

If Yes:

6. Controller → Sale (Create Sale Record)
7. Controller → Inventory (Deduct Stock)
8. Controller → AuditLog (Record Action)
9. Controller → Database (Save Changes)
10. Controller → UI (Display Confirmation)

2.6.2.5 Sequence Model Representation (Required)

Diagram



2.6.3 Process Workflow Model: Sale Execution Flow

2.6.3.1 Workflow Overview

The Activity Model represents the operational flow of the Sale process, including validation steps, decision points, alternate execution paths, and transaction completion.

The workflow reflects the sequence of actions performed during daily retail operations where Product verification and stock validation are necessary before completing a Sale.

2.6.3.2 Workflow Description

The process starts when a Staff member initiates a Sale transaction by entering Product and quantity details.

The system first validates whether the Product exists. Invalid Product entries terminate the process immediately with an error message.

If Product validation succeeds, Inventory availability is checked. Transactions requesting quantities greater than available stock are rejected to prevent incorrect Inventory values.

After successful validation:

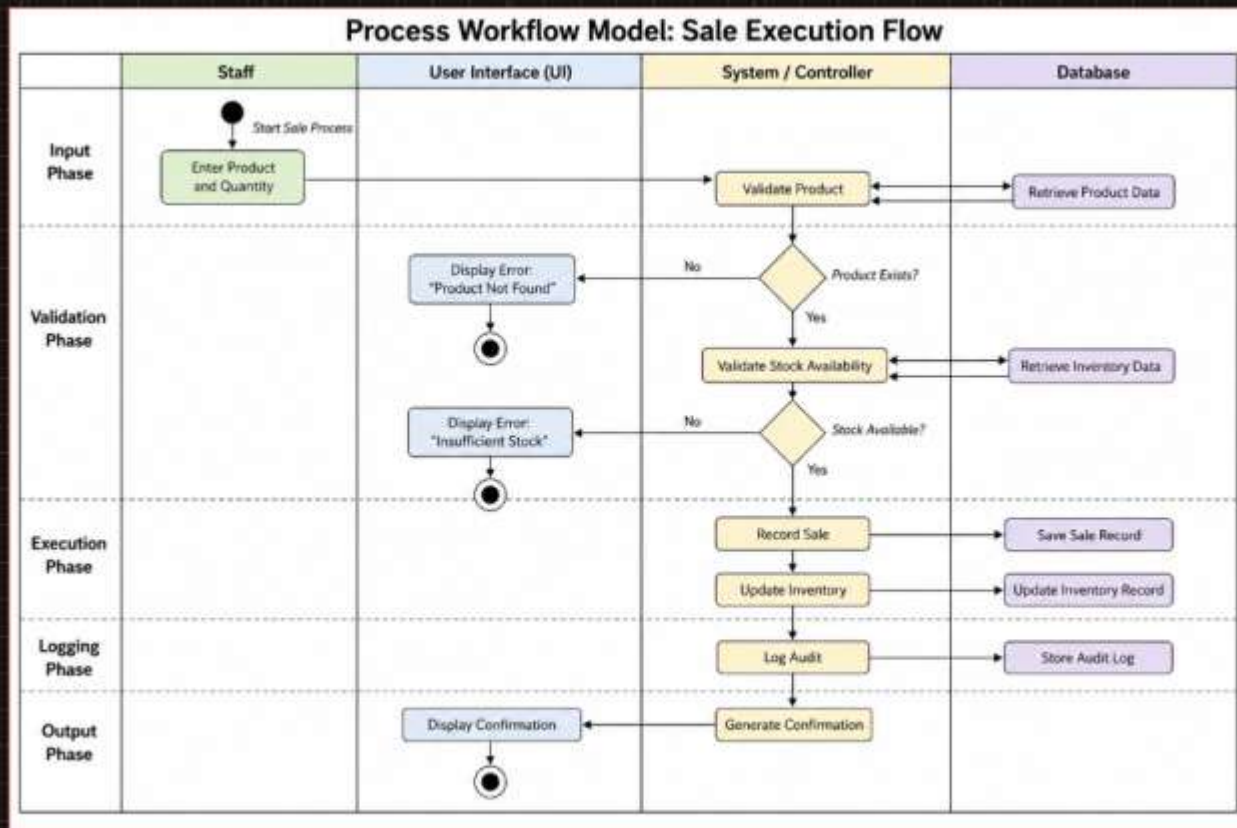
- Sale information is recorded
- Inventory quantity is updated
- AuditLog entry is created
- Confirmation message is generated

2.6.3.3 Step-by-Step Workflow

- 1) Start
- 2) Enter Product and quantity
- 3) Validate Product existence
- 4) If invalid → Display error → End
- 5) Validate Inventory availability
- 6) If insufficient → Display “Insufficient Stock” → End
- 7) Record Sale
- 8) Update Inventory
- 9) Record AuditLog entry
- 10) Display confirmation
- 11) End

2.6.3.4 Activity Model Representation:

Diagram



2.6.4 Modeling Sequence Justification

The Sequence Model is presented before the Activity Model because it explains interactions between system components and internal processing layers before illustrating workflow execution logic.

This order provides a smooth transition from interaction-level behavior to process-level workflow representation while maintaining consistency with the overall system design.

2.7 State Transition Model – Inventory and Product Stock Lifecycle

2.7.1 Introduction

The State Transition Model represents how the Inventory status of a Product changes during system operation. Unlike process-oriented models that focus on workflow execution, this model focuses on the different operational states of Inventory and the events that cause transitions between those states.

In the Smart Inventory System, Inventory behavior changes continuously as Sales are processed, stock quantities are updated, and restocking operations are performed. Modeling these transitions helps ensure that Inventory management remains consistent, accurate, and synchronized with system operations.

This model directly supports:

- FR-04 (Inventory Update)
- FR-06 (Automatic Inventory Deduction)
- FR-07 (Inventory Validation)
- FR-09 (Low Stock Alert)
- FR-10 (Threshold Management)

2.7.2 Inventory Stock States

The Inventory lifecycle is represented using the following operational states:

State	Description
Available	Product quantity exists above the defined low stock threshold and is ready for sale.
Low Stock	Product quantity falls below the predefined threshold level but is still available for sale.
Out of Stock	Product quantity becomes zero and the Product is temporarily unavailable.
Restocked	New stock has been added after a restocking operation from the Supplier.
Updated	Inventory information such as quantity or threshold has been modified by an authorized user.

2.7.3 State Transition Events

The following events trigger movement between Inventory states:

Current State	Event	Next State
Available	Sale reduces quantity below threshold	Low Stock
Available	Sale reduces quantity to zero	Out of Stock
Low Stock	Additional sales reduce quantity to zero	Out of Stock
Low Stock	New stock received	Restocked
Out of Stock	Restocking operation performed	Restocked
Restocked	Inventory validation completed	Available
Available	Inventory information modified	Updated
Updated	Update validation completed	Available

2.7.4 Validation Conditions

To maintain Inventory consistency and prevent invalid operations, the following validation conditions are applied during state transitions:

- Sale quantity must not exceed available Inventory.
- Inventory quantity cannot become negative.
- Restocking quantity must be greater than zero.
- Low Stock state is triggered automatically when quantity falls below the threshold value.
- Updated Inventory values must pass validation before returning to operational use.

These validations ensure accurate synchronization between Sales and Inventory records.

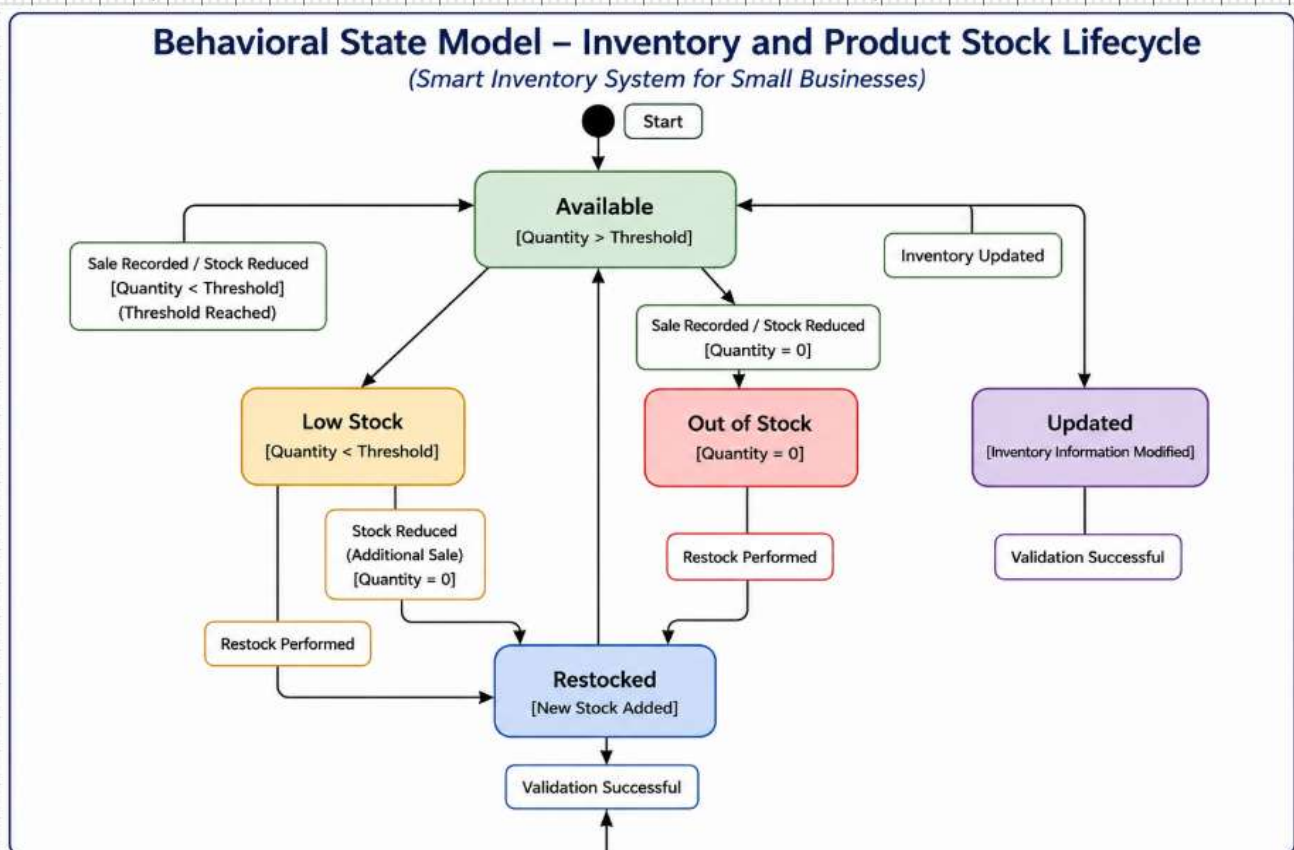
2.7.5 State Lifecycle Explanation

- The Inventory lifecycle begins in the Available state where sufficient stock exists for normal sales operations. During Sale processing, Inventory quantities decrease automatically based on the quantity sold.
- If Inventory falls below the predefined threshold, the Product enters the Low Stock state. This transition supports the low stock alert mechanism and helps shopkeepers identify Products that require restocking.
- When all stock is depleted, the Product transitions into the Out of Stock state. In this state, additional sales are prevented until new stock is received.

- After restocking operations are performed, the Product moves into the Restocked state where Inventory quantities are updated and validated. Once validation is completed successfully, the Product returns to the Available state.
- Administrative modifications to Inventory or threshold values temporarily place the Product into the Updated state before the system finalizes synchronization and validation.

2.7.6 State Transition Model Representation:

Diagram



2.7.7 Requirement Traceability

The State Transition Model maintains consistency with the previously defined Functional Requirements and Inventory workflows.

Functional Requirement	State Behavior Supported
FR-04	Inventory Update and Restocking
FR-06	Automatic Inventory Deduction
FR-07	Inventory Availability Validation
FR-09	Low Stock Alert Generation
FR-10	Threshold Management

3.0 MILESTONE 3: DESIGN AND ARCHITECTURE

3.1 System Architecture Design

3.1.1 Architecture Overview

System Architecture defines the high-level organization of the Smart Inventory System by describing how components are structured, how responsibilities are distributed, and how different layers interact during execution.

The architecture is derived from the refined UML models and operational workflows developed in previous sections.

3.1.2 Architectural Style Selection

The Smart Inventory System follows a Layered Architecture (Three-Tier Architecture) approach.

This structure separates the system into logical layers, allowing independent handling of user interaction, business logic, and data management.

3.1.3 Architectural Layers

3.2.3.1 Presentation Layer (User Interface)

The Presentation Layer manages all user interaction with the system.

❖ Responsibilities

- Accept Product, Sale, and Inventory input
- Display reports, alerts, and validation responses
- Provide navigation for operational tasks
- Handle interaction between users and system functions

3.2.3.2 Application Layer (Control Layer)

The Application Layer contains the core business logic and controls operational workflows.

❖ Responsibilities

- Validate user input
- Execute Sale and Inventory operations
- Process business rules
- Coordinate workflow execution
- Trigger AuditLog and alert operations

3.2.3.3 Data Layer (Persistence Layer)

The Data Layer handles data storage and retrieval operations.

❖ **Responsibilities**

- Store Product, Inventory, Sale, User, Supplier, and AuditLog data
- Execute database operations
- Manage record retrieval and updates
- Preserve operational data consistency

3.1.4 Layered Interaction Flow

The interaction flow between architectural layers follows this sequence:

- 1) User interacts with the Presentation Layer
- 2) Request is forwarded to the Application Layer
- 3) Business logic is processed
- 4) Data operations are performed through the Data Layer
- 5) Retrieved results return to the Application Layer
- 6) Output is displayed through the Presentation Layer

3.1.5 Justification for Layered Architecture

Layered Architecture was selected because it matches the operational scope and complexity of the Smart Inventory System.

Key reasons include:

- Clear separation between UI, logic, and data handling
- Easier development and maintenance
- Simplified testing of individual layers
- Better scalability for future extensions
- Improved readability and organization of system components
- Natural alignment with the Sequence Model workflow

3.1.7 Architectural Components

The architecture includes the following major components:

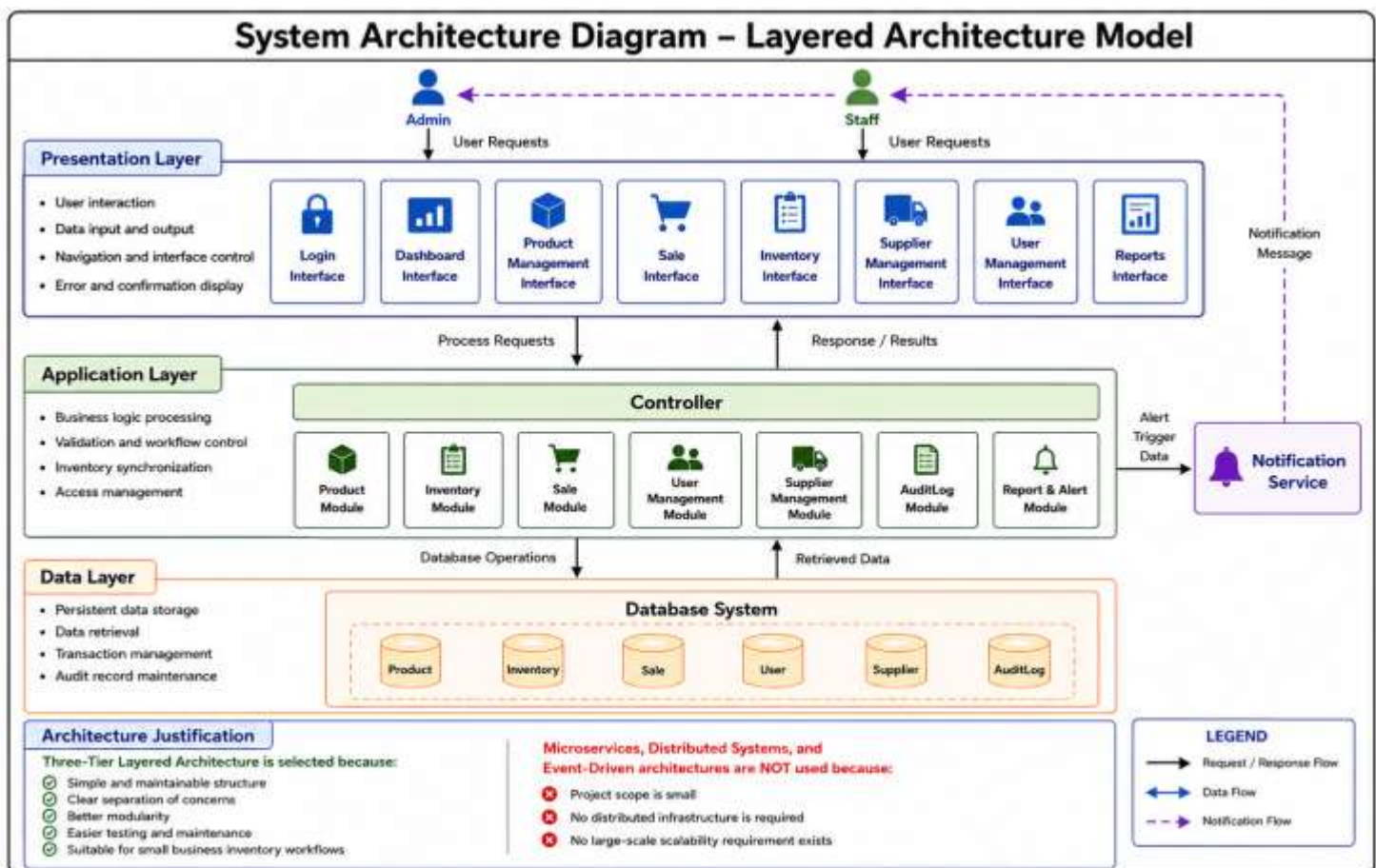
- User Interface (UI)
- Controller
- Product Module

- Inventory Module
- Sale Module
- User Management Module
- Supplier Module
- AuditLog Module
- Database System

These components correspond directly to the previously defined classes and operational workflows.

3.1.8 System Architecture Representation:

Diagram



3.2 Database Design

3.2.1 Database Design Overview

Database Design defines the relational structure used to store and manage system data. The design translates the Data Dictionary and Class Architecture into database tables, keys, and relationships.

The structure supports operational workflows such as Product management, Sale processing, Inventory updates, User control, and reporting.

3.2.2 Design Approach

The Smart Inventory System uses a Relational Database Model where data is organized into structured tables connected through defined relationships.

This approach was selected because it:

- Supports structured transactional data
- Reduces redundancy
- Preserves relationship integrity
- Simplifies querying and reporting
- Matches the system's object-oriented design

3.2.3 Core Tables and Attributes

3.2.3.1 Table: Product

Attribute	Data Type	Description
productID (PK)	int	Unique Product identifier
name	string	Product name
category	string	Product category
purchasePrice	float	Product purchase price
sellingPrice	float	Product selling price
supplierID (FK)	int	Associated Supplier ID

3.2.3.2 Table: Inventory

Attribute	Data Type	Description
-----------	-----------	-------------

inventoryID (PK)	int	Unique Inventory identifier
productID (FK)	int	Associated Product
quantity	int	Available stock quantity
lowStockThreshold	int	Minimum stock limit

3.2.3.3 Table: User

Attribute	Data Type	Description
userID (PK)	int	Unique User identifier
username	string	Login username
password	string	Encrypted password
role	string	Assigned User role

3.2.3.4 Table: Supplier

Attribute	Data Type	Description
supplierID (PK)	int	Unique Supplier identifier
name	string	Supplier name
contactInfo	string	Supplier contact details

3.2.3.5 Table: Sale

Attribute	Data Type	Description
saleID (PK)	int	Unique Sale identifier
productID (FK)	int	Sold Product
quantity	int	Quantity sold
totalPrice	float	Total transaction amount
timestamp	datetime	Sale transaction time

3.2.3.6 Table: AuditLog

Attribute	Data Type	Description
logID (PK)	int	Unique log identifier
userID (FK)	int	User performing action
actionType	string	Type of recorded action
timestamp	datetime	Action timestamp

3.2.4 Table Relationships

3.2.4.1 One-to-One Relationship

- ❖ **Product ↔ Inventory**

Each Product is associated with one Inventory record.

3.2.4.2 One-to-Many Relationships

- ❖ **Supplier → Product**

One Supplier can provide multiple Products.

- ❖ **Product → Sale**

One Product can appear in multiple Sale records.

- ❖ **User → AuditLog**

One User can generate multiple AuditLog entries.

next but ensure the correction rules

3.2.5 Integrity Constraints

The database applies multiple constraints to preserve accuracy and relationship validity across all records.

- ❖ **Primary Keys (PK)**

Primary keys uniquely identify records within each table.

- ❖ **Foreign Keys (FK)**

Foreign keys maintain valid relationships between connected tables.

- ❖ **NOT NULL Constraints**

Critical fields such as Product name, quantity, username, and role cannot remain empty.

❖ **Validation Rules**

The following validation conditions are enforced:

- Quantity values must be greater than or equal to zero
- Price values must remain positive
- Referenced foreign keys must exist before record creation

3.2.6 Database Design Alignment

The database structure remains directly connected to:

- Class Architecture
- Functional Requirements
- Data Dictionary
- Data Flow Models
- Use Cases

All tables, relationships, and attributes preserve the same entities and operational logic used throughout the project documentation.

3.3.7 Database Representation:

Diagram

3.4 Interface Design

3.4.1 Interface Design Overview

Interface Design defines how users interact with the Smart Inventory System through screens, input fields, menus, and operational controls.

The interface structure focuses on simplicity, usability, and efficient execution of daily retail operations such as Product management, Sale processing, and Inventory monitoring.

3.4.2 Interface Design Principles

The interface follows the following design principles:

❖ Simplicity

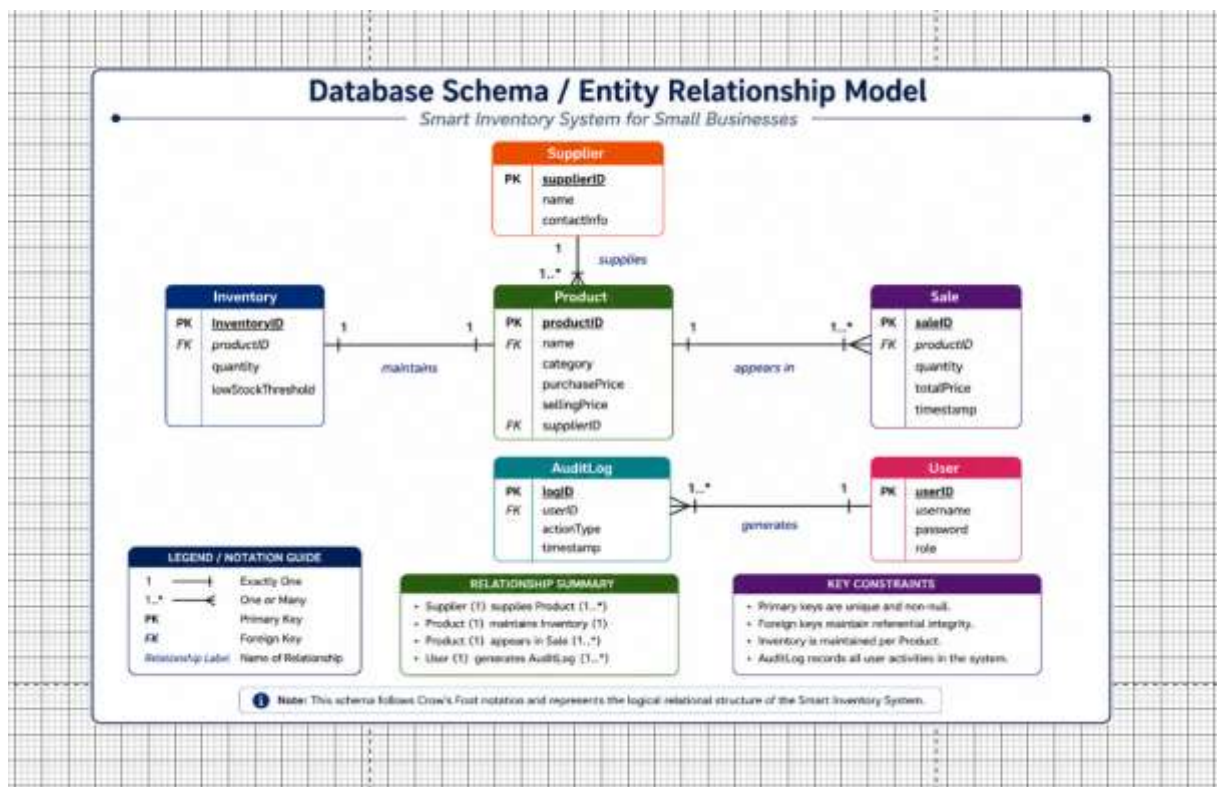
Layouts remain clear and minimal to reduce operational complexity.

❖ Consistency

Naming conventions, navigation structure, and screen organization remain uniform across the system.

❖ Feedback

Users receive immediate responses for successful operations, validation failures, and processing errors.



❖ **Error Prevention**

Input validation and controlled workflows reduce incorrect data entry.

❖ **Efficiency**

Operational tasks are designed to require minimal steps during execution.

3.4.3 User Roles and Interface Access

❖ **Admin Access**

Admin users have access to:

- Product management
- User management
- Supplier management
- Reports
- AuditLog monitoring
- System configuration

❖ **Staff Access**

Staff users can:

- Record Sales
- Update Inventory
- View stock information
- Access limited reports

3.4.4 Interface Components and Screens

3.4.4.1 Login Interface

❖ **Function**

Provides secure system access through authentication.

❖ **Input Fields**

- Username
- Password

❖ **Interface Behavior**

- Validates user credentials

- Redirects users according to assigned roles
- Displays authentication errors for invalid login attempts

3.4.4.2 Dashboard Interface

❖ Function

Provides a summary view of operational system status and quick navigation to major modules.

❖ Displayed Information

- Total Products
- Low stock alerts
- Recent Sales summary
- Inventory overview

3.4.4.3 Product Management Interface

❖ Function

Allows Admin users to manage Product records.

❖ Functionalities

- Add Product
- Update Product
- View Product details

❖ Input Fields

- Product name
- Category
- Purchase price
- Selling price
- Supplier information

3.4.4.4 Sale Recording Interface

❖ Function

Allows Staff users to process Sales transactions efficiently.

❖ **Workflow**

- 1) Select Product
- 2) Enter quantity
- 3) Validate stock availability
- 4) Confirm transaction

❖ **System Responses**

- Sale confirmation message
- Validation error for insufficient stock

3.4.4.5 Inventory Management Interface

❖ **Function**

Supports Inventory monitoring and stock updates.

❖ **Functionalities**

- View current Inventory levels
- Update stock quantities
- Display low stock alerts

3.4.4.6 Supplier Management Interface

❖ **Function**

Allows Admin users to maintain Supplier records.

❖ **Functionalities**

- Add Supplier
- Update Supplier details
- Associate Suppliers with Products

3.4.4.7 User Management Interface

❖ **Function**

Supports User account administration.

❖ **Functionalities**

- Create User accounts
- Assign User roles
- Update User information

3.4.4.8 Reporting Interface

❖ **Function**

Provides access to operational reports for monitoring and decision-making.

❖ **Displayed Reports**

- Sales reports
- Inventory reports

3.4.5 Input and Output Design

3.4.5.1 Input Design

Input handling includes:

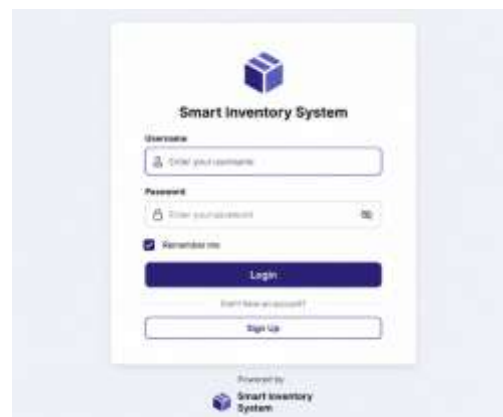
- Mandatory field validation
- Numeric validation for quantity and price fields
- Duplicate Product prevention
- Structured data entry formats

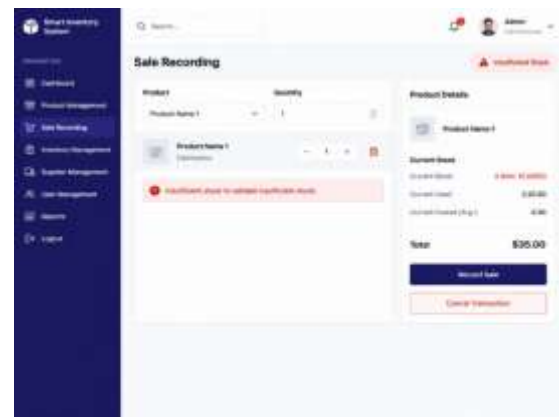
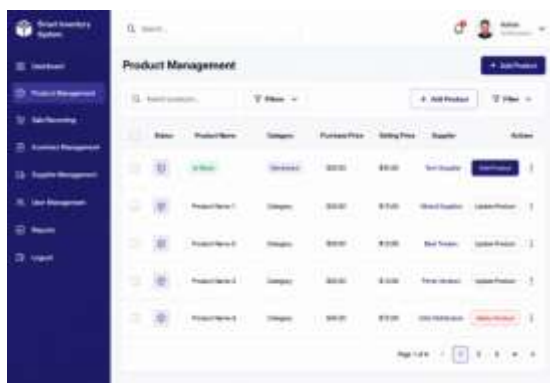
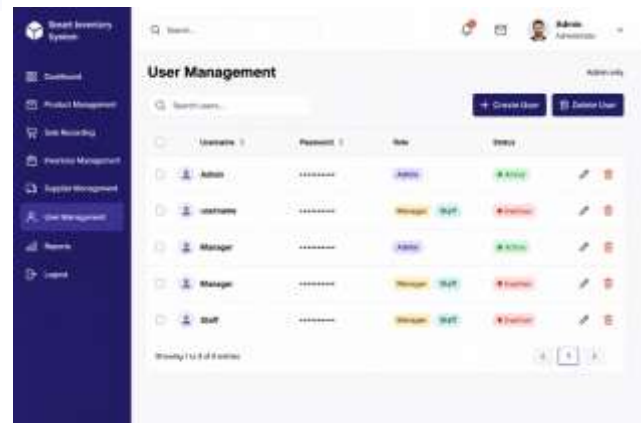
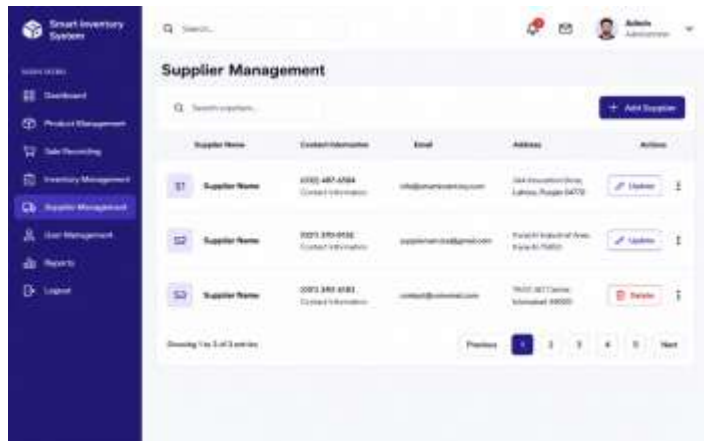
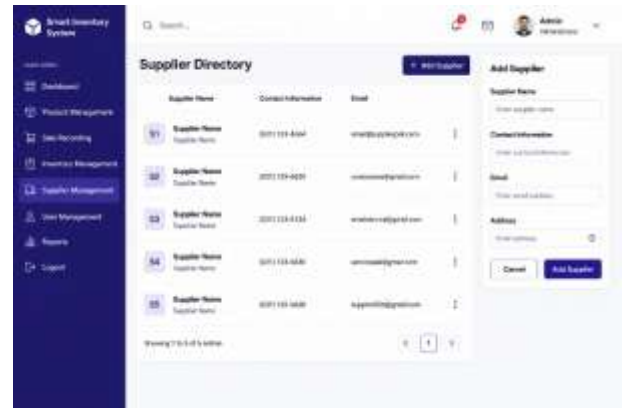
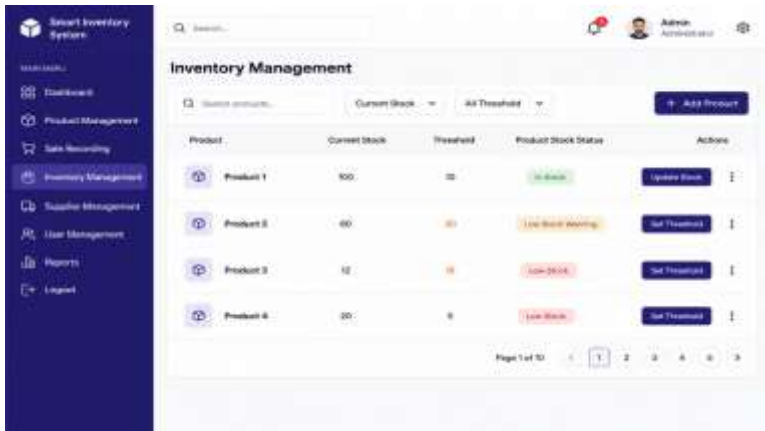
3.4.5.2 Output Design

The system provides:

- Confirmation messages for successful operations
- Clear validation and error messages
- Structured report presentation
- Real-time Inventory visibility

3.4.6 Interface Representation:





3.5 Error Handling and Validation Design

3.5.1 Error Handling and Validation Overview

Error Handling and Validation Design defines how the Smart Inventory System detects invalid input, prevents incorrect operations, and responds to exceptional conditions during execution.

The design focuses on operational reliability, accurate transaction processing, and controlled user interaction.

3.5.2 Validation Strategy

Validation is applied at multiple levels to reduce incorrect data entry and prevent invalid operational states.

3.5.2.1 Input Validation

Input validation verifies that user-provided data follows required formats and constraints before processing.

❖ Validation Rules

- Mandatory fields cannot remain empty
- Quantity and price fields must contain numeric values
- Quantity values cannot be negative
- Price values must remain positive
- Duplicate Product names are restricted

3.5.2.2 Business Logic Validation

Business logic validation ensures that operational workflows follow defined system rules.

❖ Validation Conditions

- Sale processing stops when Inventory is insufficient
- Product existence must be verified before recording a Sale
- Inventory updates occur only after successful validation
- Low stock alerts are generated only when threshold conditions are met

3.5.2.3 Authentication and Authorization Validation

Authentication and authorization validation control secure system access and permission handling.

❖ Security Validation

- Valid username and password are required before access
- User permissions depend on assigned roles

- Restricted operations remain inaccessible to unauthorized users

3.5.3 Error Handling Mechanism

The system follows a structured process for handling operational and validation errors.

3.5.3.1 Error Categories

❖ Input Errors

Generated when user input is incomplete, invalid, or incorrectly formatted.

❖ Validation Errors

Occur when business rules are violated, such as insufficient Inventory during a Sale transaction.

❖ Authentication Errors

Generated for invalid login credentials or unauthorized access attempts.

❖ System Errors

Occur during unexpected processing failures or operational interruptions.

3.5.3.2 Error Response Flow

Error handling follows this sequence:

- 1) Detect error during validation or execution
- 2) Stop the current operation
- 3) Display a clear error message
- 4) Allow the user to correct the issue
- 5) Retry operation if necessary

3.5.4 Error Message Design

Error messages are designed to remain clear, direct, and operationally useful.

❖ Example Messages

- “Product not found”
- “Insufficient stock available”
- “Invalid input: Quantity must be a positive number”
- “Login failed: Incorrect username or password”

Each message identifies the issue clearly to simplify correction and reduce operational confusion.

3.5.5 Exception Handling Workflow

The exception handling process follows this operational sequence:

- 1) User performs an action
- 2) Validation checks are executed
- 3) If validation fails → Error message displayed
- 4) If validation succeeds → Operation continues
- 5) Relevant actions or failures are recorded in AuditLog when required

3.5.6 Integration with Audit Logging

Critical operational errors and system activities may be recorded in the AuditLog for monitoring and troubleshooting purposes.

Examples include:

- Failed Sale transactions caused by insufficient Inventory
- Unauthorized access attempts
- Major Inventory modifications
- System processing failures

3.6 Security Design

3.6.1 Security Design Overview

Security Design defines the mechanisms used to protect system data, control access, and monitor critical activities within the Smart Inventory System.

The design focuses on authentication, authorization, data protection, and operational accountability.

3.6.2 Security Objectives

The system security structure is designed to:

- Prevent unauthorized access
- Protect sensitive information

- Restrict operations according to User roles
- Monitor critical system activities
- Record operational events for accountability

3.6.3 Authentication Mechanism

Authentication controls access to the system through credential verification.

3.6.3.1 Login Process

The login workflow follows this sequence:

- 1) User enters username and password
- 2) Credentials are validated against stored records
- 3) Valid credentials grant system access
- 4) Invalid credentials generate an authentication error

3.6.3.2 Credential Protection

Credential security includes:

- Encrypted password storage
- Restricted access to authentication data
- Secure comparison during login validation

Plain-text password storage is not permitted.

3.6.4 Authorization and Access Control

Authorization controls which operations each User role can access.

3.6.4.1 Role-Based Access Structure

❖ Admin

Admin users can:

- Manage Products
- Manage Users
- Manage Suppliers
- View reports
- Access AuditLog information

- Configure system settings

❖ **Staff**

Staff users can:

- Record Sales
- Update Inventory
- View stock information
- Access limited reports

3.6.4.2 Access Restrictions

- Unauthorized operations are blocked automatically
- Administrative functions remain restricted to Admin users
- Permission checks are performed before operation execution

3.6.5 Data Protection Mechanisms

3.6.5.1 Data Encryption

Sensitive information such as User credentials remains encrypted during storage.

3.6.5.2 Data Integrity Protection

Database constraints and controlled transaction handling reduce invalid updates and incomplete processing states.

3.6.6 Audit Logging for Security

Audit logging supports operational monitoring and activity tracking.

❖ **Logged Activities**

The following activities may be recorded:

- Product creation and updates
- Inventory modifications
- Sale transactions
- User management actions
- Login attempts

❖ **Security Monitoring Benefits**

Audit logging helps:

- Track user activities

- Detect unauthorized operations
- Monitor operational history
- Support troubleshooting and investigation

3.6.7 Security Constraints

The following security constraints are enforced throughout the system:

- Authentication is required before system access
- User permissions follow assigned roles
- Critical activities are recorded in AuditLog
- Sensitive information remains protected through encryption

References

1. Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
2. Roger S. Pressman & Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill, 2019.
3. Kenneth E. Kendall & Julie E. Kendall, *Systems Analysis and Design*, 10th Edition, Pearson, 2019.
4. Alan Dennis, Barbara Haley Wixom & Roberta M. Roth, *Systems Analysis and Design*, 8th Edition, Wiley, 2020.
5. Grady Booch, James Rumbaugh & Ivar Jacobson, *The Unified Modeling Language User Guide*, 2nd Edition, Addison-Wesley, 2005.
6. Thomas Connolly & Carolyn Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th Edition, Pearson, 2015.

Tools Used

- draw.io
- Canva
- Microsoft Layered Architecture Documentation
- Lucidchart ER Diagram Guide

- OMG UML Specification
- Microsoft Word
- UML Modeling
- DFD Modeling
- Crow's Foot Notation
- Three-Tier Layered Architecture
- Object-Oriented Design (OOD)
- Relational Database Design
- Requirement Traceability Matrix (RTM)